

Figure 12: Finite State Machine (FSM) Example Implementation in LabVIEW

Producer-Consumer LabVIEW Design Pattern

The second paradigm employed, upon which both **main.vi** and the subVI **Arduino_FlowPump_Controller.vi** are based, is the **Producer-Consumer (Event-Driven)** design pattern [7]. This approach is particularly well-suited for real-time applications that must respond promptly to user-generated events through the graphical user interface (GUI), while simultaneously ensuring that the program remains non-blocking—that is, each event is handled asynchronously and independently from others.

This architecture consists of an event-handling structure, referred to as the 'producer', which monitors all relevant user-GUI interactions based on the software's functional requirements. The producer defines the behavioral response for each incoming event, but is unaware of the actual implementation details necessary for event handling. These are delegated to a separate structure known as the 'consumer'. Once the consumer receives a command from the producer, it executes the corresponding operations necessary to complete the event-handling cycle.

Figure 13 illustrates the block diagram implementing a finite state machine alongside the event-based Producer-Consumer pattern. The events correspond to user-GUI interactions. Each relevant interaction triggers a state transition within the finite state machine, which is implemented in the consumer loop using an enumerated constant to manage state flow.

In this project, a multi-consumer architecture was adopted, where each consumer is designed to respond to events of a different "nature"—that is, events that require access to different software or hardware resources.

Arduino UNO R3

5 Connector Pinouts

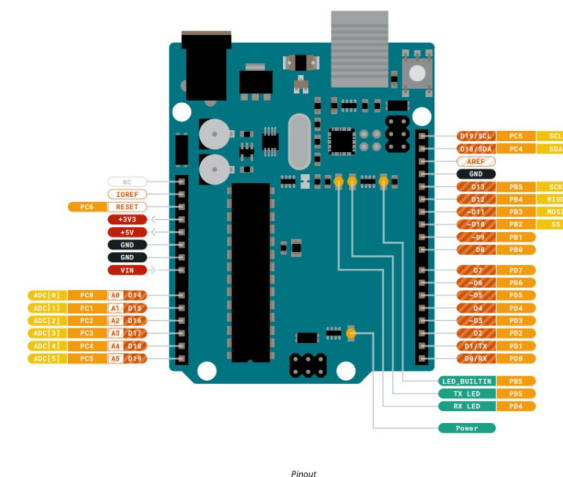


Figure 6: Arduino UNO R3 Pins [5]

Arduino is an open-source platform encompassing a programmable electronic board and an Integrated Development Environment (IDE). The IDE facilitates writing, compiling, and uploading C/C++ based programs (sketches) to the board, leveraging simplified custom libraries and functions like **digitalWrite()** and **analogRead()**, along with a Serial Monitor for real-time communication. A typical Arduino Uno board integrates a central **Microcontroller Unit (MCU)** (e.g., ATmega328P) with built-in memory (Flash, SRAM, EEPROM) to execute code and manage I/O. Its diverse pins include **Digital Input/Output (I/O) Pins** (0–13) for digital control, **PWM Pins** (e.g., 3, 5, 6, 9, 10, 11) for analog simulation, and **Analog Input Pins** (A0–A5) connected to a 10-bit Analog-to-Digital Converter (ADC) for reading variable voltages (0–1023). Essential **Power Pins** (5V, 3.3V, GND, VIN) provide regulated power, while a **USB Port** enables PC connection for code upload, power, and serial communication. A **Reset Button** restarts the MCU, and a **Voltage Regulator** ensures stable internal power. In the context of this Project's work, the Arduino UNO R3 board was used to acquire the digital flow transducer [3] measurements, used as a reference for the calibration procedure. The script that must be installed on the microprocessor is described in the *Appendix A*.

Signal Conditioning Circuit

Circuit Design

The signal conditioning circuit was specifically engineered to amplify the output signals from the analog sensor [2] under calibration. These signals are inherently low-amplitude, rendering them unsuitable for precise detection of airflow variations within the sensor working range. The sensor provides two distinct outputs: a voltage generated by thermopile TP2, exposed to unheated airflow, and a voltage from thermopile TP1, subjected to a temperature variation. The latter exhibits a voltage variation (approximately 15mV at maximum input flow range, as per datasheet specifications) induced by a strategically placed resistor between the two thermopiles. Upon heating, this resistor elevates the temperature of the airflow directed towards the sensor's output, where thermopile TP1 is situated. Consequently, an electrical potential component, attributable to the Seebeck effect, causes a voltage deviation from the no-flow condition.

Nevertheless, experimental measurements using a multimeter corroborate the sensor's datasheet specifications, indicating that the voltages from both thermopiles are in the range of hundreds of millivolts.

Furthermore, to mitigate the risk of overheating the current-controlled air pump, we elected to operate significantly below the sensor's maximum input range, limiting the flow to a maximum of 6 L/min (compared to a specified maximum of 10 L/min). This operational constraint implies that the observed voltage variation will be substantially less than the 15mV indicated in the datasheet.

In essence, the available signals are insufficient to accurately discriminate airflow passage and thus

Before any modifications can be made to the program, the reader should be familiar with the concepts of *Enum*, *Queue Operations*, *Event Case Structure*, and *Finite State Machines* as implemented in LabVIEW. Therefore, a brief overview of some of the software paradigms used in this project will be provided, with particular focus on Finite State Machines (FSM) and the Producer-Consumer design pattern.

Finite State Machine (FSM) LabVIEW Design Pattern

A **Finite State Machine** (or FSM) is a computational model used to represent a system that can be in only one state at a time, and that transitions from one state to another in response to specific events or conditions. It is one of the most commonly used paradigms for designing control logic, operational sequences, and reactive systems.

In LabVIEW, a finite state machine can be implemented in various ways [6]. For this project, the FSM has been implemented using a queue-based state scheduling mechanism [7], made possible through the **Synchronization/Queue Operations sub-palette**. Figure 12 shows the architecture adopted for all FSMs within this project. Each FSM consists of a **while loop** that iteratively executes the machine, a **Guard Clause**—a case structure that handles potential errors during execution—and a second case structure responsible for decoding and executing the current state's actions, as well as managing the scheduling of the next state.

Outside the while loop of a general Queued-Finite State Machine implementation in LabVIEW, as illustrated in Figure 12, a queue is initialized and used to post state transition commands. The queue's data type is defined as an **Enum** constant, which maps integer values to string labels, thereby abstracting the machine's states into human-readable representations.

The subVIs implementing this FSM paradigm are **Agilent_GPIB_FSM.vi** and **Arduino_Serial_FSM.vi**, which control the logical sequencing and operations required to remotely manage communication with the **Arduino UNO** board and the **Agilent e3634A** power supply, respectively.

Software



The LabVIEW based Virtual Instrument

The study of the components of the measurement system, along with the delicate task of managing the acquisition of measurement samples simultaneously from multiple devices through different instrument interfaces, led to the choice of the **LabVIEW** development environment. Graphical programming in LabVIEW greatly facilitates the programmer's task, thanks to the wide selection of libraries and software extensions, which offer a high-level abstraction compared to low-level programming of interfaces such as **GPIO** or the **Serial** interface. Furthermore, LabVIEW is a programming environment that was specifically designed to handle the acquisition and generation of input and output signals, respectively, through **DAQ** data acquisition boards.

For these reasons, in order to develop a software system capable of managing the various instruments present in the physical measurement system, we developed several applications (or **Virtual Instruments (VI)**), each tasked with handling communication with a given device through different programming interfaces.



Figure 11: Icons representing different components (VIs) of the main developed software.

obtain reliable measurements.

Given these limitations, the implementation of a differential amplifier became necessary.

The instrumentation amplifier - INA128P

Fundamentally, the raw signals available are inadequate for reliably discriminating airflow and thus obtaining accurate measurements.

Therefore, the integration of a differential amplifier was deemed essential. We specifically selected the Texas Instruments INA128P differential amplifier.

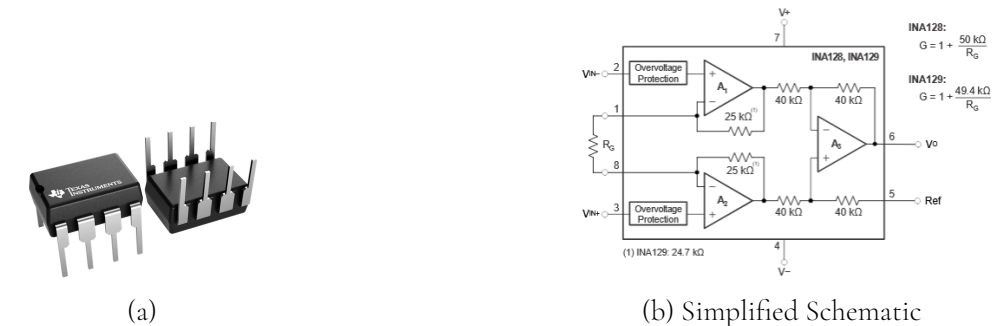


Figure 7: INA128P package (a) and its corresponding simplified schematic (b).

The INA128 is a low-power, precision instrumentation amplifier, ideally suited for measurement applications demanding high accuracy, minimal drift, and excellent common-mode rejection. Its integrated three-operational amplifier design facilitates a wide range of configurable gains via a single external resistor, thereby ensuring exceptional versatility. Key performance characteristics contributing to the device's efficiency include a low offset voltage (maximum 50 μV), low quiescent current consumption (700 μA), and an outstanding Common Mode Rejection Ratio (CMRR) whose minimum values is 120dB.

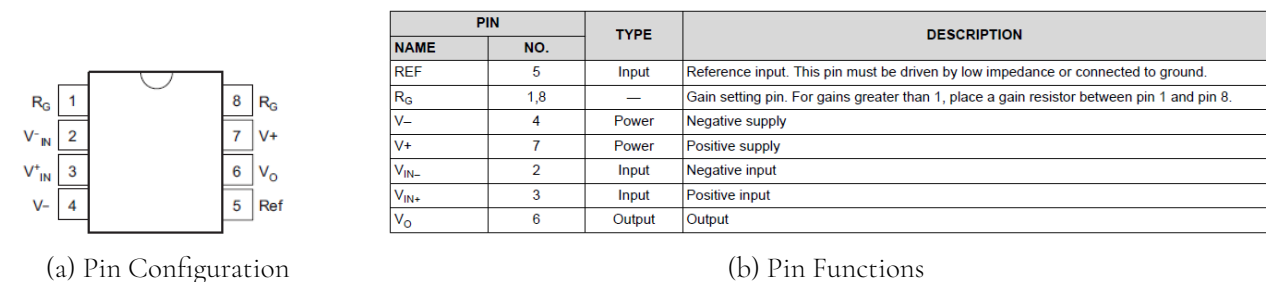


Figure 8: Pin INA128P

Regarding pin configuration, according to the information reported in the component’s datasheet in Figure 8, terminals 1 and 8 are designated for connecting the gain resistor; terminal 7 receives the power supply; the output signal is drawn from terminal 6; terminals 4 and 5 are connected to a common ground; and terminals 3 and 2 are connected to the signals from which the difference is to be taken.

The gain factor is calculated using the formula provided in the datasheet, where the amplifier’s internal feedback resistance is fixed (50 k Ω), while the gain resistance can be varied by connecting an appropriately valued resistor to terminals 1 and 8. During the experimental phases, the gain resistance was set using a potentiometer to the desired value, according to the gain table in the component’s datasheet.

Potentiometer

The designed circuit incorporates a Bourns 3296W-1-103LF multi-turn precision potentiometer, employed for fine-tuning the gain of the differential amplifier. The potentiometer is configured as a resistive divider, with its central terminal (also known as wiper) connected to the amplifier’s feedback node. Rotating the trimmer clockwise increases the resistance between the wiper and the CCW terminal, thereby adjusting the gain based on the input signal conditions. Through the potentiometer, the gain resistance was set to 100 Ω , with a tolerance of $\pm 10\%$ as per the device’s datasheet. This configuration corresponds to a gain factor of 500. Consequently, if the initial difference between the two signals varies by a few millivolts, the resulting amplified signal will vary between a few hundred millivolts. The potentiometer was connected to the pins 1 and 8 of the used INA128P device.

Devices Interconnections

The conditioning circuit is connected to the measurement system instruments via jumper wires. These interconnections are fully explained in the *User’s Manual*. Hence, please refer to that document for more information. In this section, simple illustrations are provided to the reader, offering a detailed look at an experimental setup. Figure 9a shows the power supplies, stacked one on top of the other. Next, in Figure 9b, the back of the design Conditioning Circuit is shown. This view reveals the complexity of the soldered connections, which are essential for processing and preparing the signals from the sensors before they can be interpreted or used by the system. Then, Figure 10a presents an overview of the

automated measuring system’s hardware setup. Here we see the conditioning board mounted on a wooden base, connected via cables to the other components, showing how the various elements are physically integrated. Finally, Figure 10b illustrates the electrical schematic of the whole measuring system, where the only element missing is the PC or workstation used and its connections.

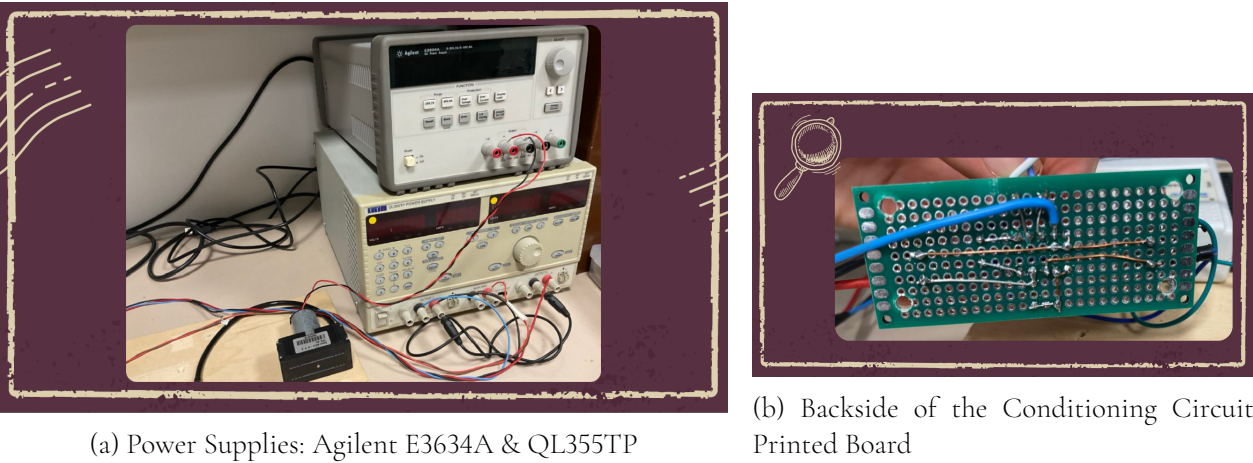


Figure 9: Experimental setup components: Power supplies and the backside of the conditioning circuit board.

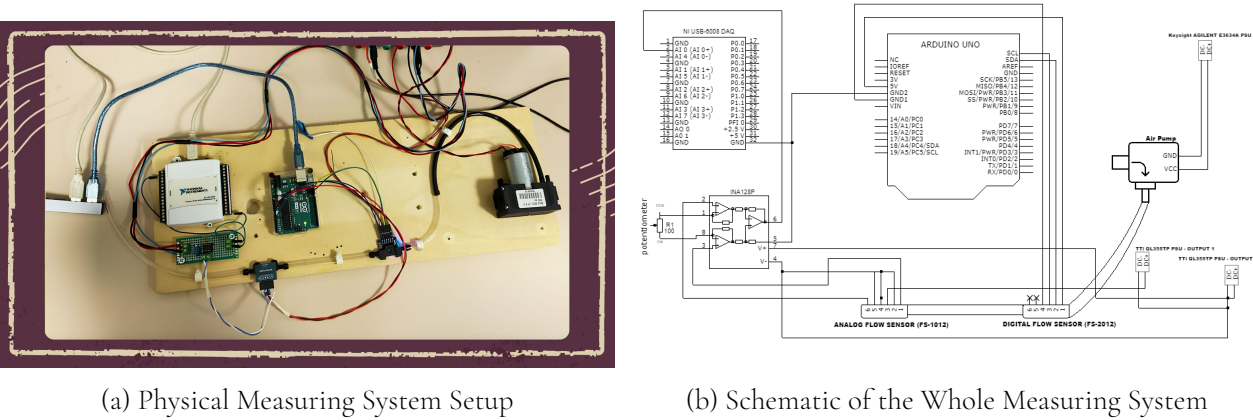


Figure 10: Overview of the measuring system: Physical setup (left) and detailed schematic (right).